

Bloom Filters

Hoa Vu
CS 663

SDSU

March 1, 2026

The Membership Problem

- ▶ **Goal:** Given a set S , answer queries “is $x \in S$?”
- ▶ **Goal:** S is the set of malicious URLs.
- ▶ **Exact solution:** hash table — uses $\Theta(|S| \times \text{item size})$ words of space.
- ▶ A url of length 200 characters would be 200 bytes. For $|S| = 10^7$, this is 1.8 GiB of memory.

The Membership Problem

- ▶ **Can we do better?** Use $O(|S|)$ *bits* instead of words.
- ▶ For $|S| = 10^7$, this would be only 1.5 MiB of memory.
- ▶ **Applications:** spell checkers, network routers, database joins, web caches.

The Trade-off

A Bloom filter uses far less space by allowing **false positives** and not having to store the actual elements.

- ▶ **False positive:** filter says “yes” but $x \notin S$.
- ▶ **False negative:** *never happens*. If $x \in S$, filter always says “yes”.

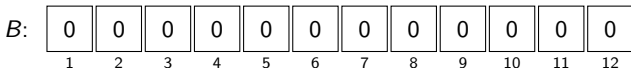
We control the false positive rate ε by choosing filter parameters.

Data Structure

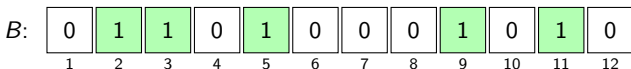
- ▶ $|S| \leq n$
- ▶ m -bit array $B[1 \dots m]$, initially all **0**.
- ▶ k independent hash functions $h_1, \dots, h_k : \mathcal{U} \rightarrow \{1, \dots, m\}$.
- ▶ **Insert**(x): For $i = 1$ to k : set $B[h_i(x)] \leftarrow 1$.
- ▶ **Query**(x): Return **yes** iff $B[h_i(x)] = 1$ for *all* i .

Visualization: Insertion

Insert x and y with $k = 3$ hash functions, $m = 12$ bits.



After inserting x ($h_1 = 2, h_2 = 5, h_3 = 9$) and y ($h_1 = 3, h_2 = 5, h_3 = 11$):



► Bit 5 is shared by both x and y (**collision**).

Visualization: True Negative

Query z with $h_1(z) = 3$, $h_2(z) = 7$, $h_3(z) = 11$:

B :

0	1	1	0	1	0	0	0	1	0	1	0
1	2	3	4	5	6	7	8	9	10	11	12

- ▶ Check positions 3, 7, 11: $B[3] = 1$, $B[7] = 0$, $B[11] = 1$.
- ▶ $B[7] = 0 \Rightarrow$ answer is **no**. Correct: z was never inserted.

Visualization: False Positive

Query z' with $h_1(z') = 2$, $h_2(z') = 5$, $h_3(z') = 9$:

B :

0	1	1	0	1	0	0	0	1	0	1	0
1	2	3	4	5	6	7	8	9	10	11	12

- ▶ Check positions 2, 5, 9: all equal **1** $\Rightarrow$ answer is **yes**.
- ▶ But z' was *never inserted* — this is a **false positive**.
- ▶ The bits were set by x , not by z' .

Probability a Bit is Zero

After inserting m elements using k hash functions into m bits:

Each insertion sets a uniformly random bit in each row. Use the approximation $1 - x \approx e^{-x}$ for small x . The probability a specific bit $B[j]$ is **never set**:

$$\Pr[B[j] = 0] = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$$

► More insertions $\Rightarrow$ fewer zero bits $\Rightarrow$ more false positives.

False Positive Probability

A false positive occurs when *all* k probed bits equal 1.

Each probed bit is 1 with probability $1 - e^{-kn/m}$, independently:

$$p_{\text{fp}} \approx \left(1 - e^{-kn/m}\right)^k$$

► Set $k = \log_2(1/\varepsilon)$ and $m = n \log_2(1/\varepsilon) / \ln 2$.

$$p_{\text{fp}} \approx \left(1 - e^{-\ln 2}\right)^{\log_2(1/\varepsilon)} = \left(\frac{1}{2}\right)^{\log_2(1/\varepsilon)} = \varepsilon$$

False Positive Probability

A false positive occurs when *all* k probed bits equal 1.

Each probed bit is 1 with probability $1 - e^{-kn/m}$, independently:

$$p_{\text{fp}} \approx \left(1 - e^{-kn/m}\right)^k$$

► Set $k = \log_2(1/\varepsilon)$ and $m = n \log_2(1/\varepsilon) / \ln 2$.

$$p_{\text{fp}} \approx \left(1 - e^{-\ln 2}\right)^{\log_2(1/\varepsilon)} = \left(\frac{1}{2}\right)^{\log_2(1/\varepsilon)} = \varepsilon$$

Space Requirement

- Required: $m = O\left(n \log_2 \frac{1}{\varepsilon}\right)$ bits.

Space vs. False Positive Rate

optimal k^*	false positive rate
3	$\approx 9.2\%$
6	$\approx 2.2\%$
7	$\approx 0.82\%$
11	$\approx 0.046\%$

- Doubling m/n roughly **squares** the false positive rate.

Space and Time

- ▶ **Space:** $m = O(n \log(1/\varepsilon))$ bits.
- ▶ **Insert:** $O(k)$ hash evaluations.
- ▶ **Query:** $O(k)$ hash evaluations.
- ▶ With $k = \log_2(1/\varepsilon)$: both operations take $O(\log(1/\varepsilon))$ time.

Summary

- ▶ Bloom filter: approximate set membership in $O(n \log(1/\varepsilon))$ **bits**.
- ▶ **No false negatives**; false positive rate $\leq \varepsilon$.
- ▶ Optimal: $k = \log_2(1/\varepsilon)$ hash functions, half the bits remain zero.
- ▶ **No deletions** — use a Counting or Cuckoo Filter for that.